

Article

New Ant Colony Optimization Algorithms for Variants of Multidimensional Assignments in d-Partite Graphs

Krzysztof Schiff 

Faculty of Electrical and Computer Engineering, Cracow University of Technology, 31-155 Cracow, Poland; kschiff@pk.edu.pl

Abstract

This article presents ant algorithms for single- and multi-criteria industrial optimization problems. A common factor in these algorithms is the determination of the set with the maximum number of cliques, which represent the solution to multidimensional assignment problems in d-partite graphs. In the case of weighted incomplete graphs, the goal is to determine the set with the maximum number of cliques and the maximum sum of the weights of their edges. In the case of unweighted incomplete graphs, the goal is to determine the set with the maximum number of maximum cliques. In the case of complete weighted graphs, the goal is to determine all maximum cliques with the minimal sum of their edge weights. These optimization problems are solved using the various ant algorithms proposed in this paper. The proposed algorithms differ not only in terms of the objective function, but also in terms of desirability functions, as previously established, and they achieved a smaller sum of weights for cliques in the case of weighted complete graphs than previous ant algorithms presented in the literature. The same applies to unweighted incomplete graphs. The presented algorithms resulted in a greater number of maximal cliques than previous ant algorithms presented in the literature. This study is the first to propose the presented ant algorithms in the case of weighted incomplete graphs.

Keywords: ant algorithms; multidimensional assignment; maximum clique; d-partite graph; multi-criteria optimization; industrial control



Academic Editor: Paolo Renna

Received: 21 June 2025

Revised: 15 July 2025

Accepted: 20 July 2025

Published: 24 July 2025

Citation: Schiff, K. New Ant Colony Optimization Algorithms for Variants of Multidimensional Assignments in d-Partite Graphs. *Appl. Sci.* **2025**, *15*, 8251. <https://doi.org/10.3390/app15158251>

Copyright: © 2025 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

A bibliographic study concerning the problems considered in this study showed that no ant colony algorithms have been published yet to address the problem of determining the maximum number of cliques with the maximum sum of their weights in d-partite incomplete weighted graphs; however, in this paper, such an ant algorithm is presented. To address the problem of determining all maximum cliques with the minimum sum of their weights in complete weighted graphs, solutions for real-world applications were presented in [1–4]. Studies by the same author [1–3] investigated problems of small sizes, up to seven vertices. Meanwhile, the authors of [4,5] demonstrated the weakness of the algorithms presented in [1–3] against problems of large sizes while presenting other formulations of the objective function. The problem of determining the maximum number of maximum cliques in unweighted incomplete graphs was addressed in [5]. In [4,5], the effects of different forms of desire functions on the obtained results considering the look-ahead procedure was not studied. New ant algorithms for the above two problems are presented in this paper that allow us to obtain a smaller sum of weights of the maximum cliques and a greater number of maximum cliques. The problem of determining all maximum cliques

in complete and incomplete graphs can be presented as a multidimensional assignment problem [1,2]. This problem is NP-hard [6]. The multidimensional assignment problem can be encountered in many industrial applications, for example, in vision systems [7–10] or in control problems that occur in textile production systems [11,12].

To solve the problem of searching for the maximum number of cliques with the minimum sum of their weights in d-partite graphs the General Maximum-Minimal Clique Problem (GMMCP) trace tool was recently proposed [7,13]. The problem of assigning measurements to objects can be transformed to a 0–1 optimization problem, where the total distance/benefit of assigning targets to measurements is minimized/maximized [14,15]. The multidimensional assignment problem is NP-hard [16–18].

Some heuristic algorithms have been developed to solve these problems [19–21]. In [19], data association is used, which incorporates both motion and appearance and solves the data association problem for one object at a time. Tracking multiple targets is achieved through the minimization of a continuous energy function. The global energy function depends on all vertices in all parts of the graph. Object identification is carried out based on a single frame by associating data with the corresponding object [20,21].

In [15], recursive algorithms using the Hungarian method were studied. In [22], the problem was constructed as a network flow graph with cost edges. All edges are assigned weights with different costs depending on the degree of connection between the vertices, so the minimum-cost flow solution involves globally finding optimal paths for tracking multiple objects. The multi-target tracking problem in frame-by-frame detections was reduced to standard linear programming, and the problem was solved by the k-shortest path algorithm [23]. The data association problem is mapped to a cost flow network, and the optimal object detection is found by a min-cost flow algorithm in the network [24]. The authors of [25] presented the multi-target tracking problem as a flow network, and a solution was found using the min-cost flow algorithm. In [26], a dynamic version of the successive shortest path algorithm was introduced, which solves the multi-target tracking problem formulated as a min-cost flow problem. In [27], multi-object tracking was formulated as a min-cost flow optimization problem and solved by an algorithm that incorporates quadratic pairwise costs into the min-cost flow network. The problem can thus be expressed as an integer quadratic program. In [28], the multi-target tracking problem was formulated as an evidence graph. “Negative” evidence between two vertices means that the vertices represent two different objects, and “positive” evidence means that the vertices represent the same object or two similar objects. Multi-person tracking is then transformed into a graph partitioning problem. In [29], the problem of tracking multiple targets was formulated as a Minimum-Cost Subgraph Multi-Cut Problem and compared to a problem formulated as a Minimum-Cost Disjoint Path Problem. In [30], the multi-target problem was transformed into the set packing problem, solved through 0-1 integer programming, and the set partitioning problem, solved through 0–1 integer programming.

Neural networks were used in [31,32]. Meanwhile [33] first determines the appropriate number of vertices; then, the edge weights are estimated using the Kalman filter, and these weights are further improved by applying an artificial neural network. Finally, ref. [34] uses a genetic algorithm that employs fuzzy logic techniques to dynamically determine the probability of mutation.

This paper is organized as follows: in Section 2, the mathematical basis of this study is presented; in Section 3, the ant algorithms are described; in Section 4, different desire functions are introduced; Section 5 presents tables containing the data obtained during the experiments carried out using different ant colony optimization algorithms and with

different input parameter values for three of the above-mentioned problems; and the paper ends with our conclusions.

2. Mathematical Basis

A graph with d parts ($d \geq 2$) is a graph whose set of vertices can be divided into d disjoint subsets $V_1, \dots, V_d$, such that there is no edge connecting vertices belonging to the same subsets V_i , $V(G) = \bigcup_{i=1 \dots d} V_i(G)$, and such that $\forall u, v \in V_i \{u, v\} \notin E(G)$. The graph $G_{d,m} = (V, E, W)$ can be partitioned into m maximum cliques K_i ($i = 1, \dots, m$), where m is the number of vertices in V_i . If there are d parts of the graph, then each of the maximum cliques K_i should have d vertices, where $v_i \in V$ and $i = 1, \dots, d$. Each edge where $e_{ij} \in E$ has its own weight $w_{ij} \in W$, and each edge e_{ij} connects two vertices, v_i and v_j , which belong to different parts of the graph, $i = 1, \dots, m, j = 1, \dots, m$. Each clique K_i has $d * (d - 1)$ edges, so for each clique, one can compute the sum of the weights of all of its edges. There are m maximal cliques. However, if the graph does not have all of the edges, it is considered noncomplete; then, there are no m maximum cliques; their number is less than m , but this number is not known. If the graph does have all of the edges, it is considered complete. The partitioning of a weighted graph with m vertices in each of its parts into m maximum cliques with a minimum total weight can be represented as a multidimensional assignment problem (i.e., matching elements from $V = \{V_1, \dots, V_d, d > 2\}$ disjoint sets of equal size d with the smallest total weight) and can be formulated as a binary integer problem. The formulas expressing the multidimensional assignment problem are as follows:

$$\min \sum_{i1 \in V1} \dots \sum_{id \in Vd} c_{i1,i2,id} X_{i1,i2,id} \quad (1)$$

$$\begin{aligned} & s.t. \sum_{i2 \in V2} \dots \sum_{im \in Vd} X_{i1,i2,\dots,id} = 1, i_1 \in V_1 \\ & \sum_{i1 \in V1} \dots \sum_{is-1 \in Vs-1} \sum_{s+1 \in Vs+1} \dots \sum_{id \in Vd} X_{i1,i2,id} = 1, i_s \in V_s, s = 2, \dots, d-1 \end{aligned} \quad (2)$$

$$\sum_{i1 \in V1} \dots \sum_{id-1 \in Vd-1} X_{i1,id} = 1, i_d \in V_d, X_{i1,i2,id} \in \{0, 1\}, i_s \in V_s, s = 1, \dots, d \quad (3)$$

where for every d -vertex clique $(i_1, i_2, \dots, i_d) \in V_1 \times V_2 \times \dots \times V_d$, the variable $x_{i1,i2,\dots,id}$ takes the value of one if the elements of this clique belonging to the partitioned graph; otherwise, it is zero. The total cost of partitioning into m cliques with d vertices (Equation (1)) is calculated as the cost of matching elements from different sets V_i . For example, a clique with d elements $(i_1, i_2, \dots, i_d)$ would cost $w_{i1,i2,\dots,id}$.

$$w_{i1,i2,\dots,id} = \sum_{s=1}^d \sum_{t=s+1}^d w_{isit} \quad (4)$$

3. The Multidimensional Assignment Problem in d-Partite Graphs

The problem of partitioning a partial graph with m vertices in each part can be presented as a multidimensional assignment problem. A d -partite graph with m vertices in each part is shown at Figure 1, where the number of graph parts d is four and the number of vertices in each part of graph m is four. One maximal clique is presented in Figure 1, indicated by a solid black line. We have four maximal cliques if the graph has all edges between all pairs of vertices belonging to different parts of the graph.

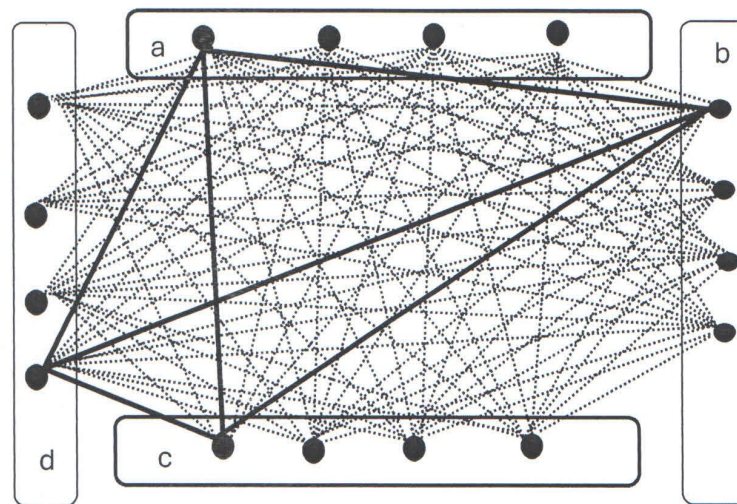


Figure 1. One maximal clique in 4-part graph.

4. Ant Algorithms

The ant colony optimization algorithm is the computational equivalent of ant swarm behavior. Ants employ optimization to find the shortest paths to food. These paths, i.e., the optimal solutions, are marked by the ants with a pheromone, which is used to communicate the path's location to the other ants. In this way, they search for the optimal solution.

The input for the ant algorithm is a graph consisting of d separate parts and m vertices in each part of the graph, as well as weights assigned to the edges. The ant algorithm is executed in two main loops: the first loop is executed as many times as the assumed number of cycles lc , and the second loop is executed as many times as the assumed number of ants lm . The d -partite graph $G_{d,m}$ is represented by the array $[V_i][m][V_j][m]$, $i \neq j$, and $i, j \leq d$.

Ants communicate with each other using the pheromone τ_{ij} , which is placed on all edges e_{ij} . In each cycle, the pheromone evaporates at a rate determined by the pheromone evaporation coefficient ρ , and an additional amount of pheromone $d\tau$ is placed on the edges of the maximal cliques, forming either the solution of all maximal cliques with the smallest sum of their weights (Algorithm 1) or the solution of all maximal cliques with the highest number and simultaneously the maximal sum of their weights (Algorithm 2). The amount of additional pheromone depends on this total weight W_K and has different functions in these two ant algorithms depending on the optimization goal. The total weight W_K is the sum of the weights of all maximum clicks determined by a single ant in the cycle. They are different for the minimization and maximization functions.

The edges that make up the best solution in a given cycle make up all the maximum cliques of the solution with the lowest total weight. These all maximal cliques constitute a set referred to as *set_of_max_cliques*. The cost W_K indicates the smallest total weight of all maximum cliques that are obtained as the best solution by all ants in a single cycle. Each ant defines the maximum number of cliques consisting of d vertices, such that whenever vertex i from some part of the graph is incorporated into the clique being formed, vertex j from the other part of the graph can be incorporated into this clique with the probability p_{ij} . The probability p_{ij} depends on the amount of pheromone τ_{ij} on the edge e_{ij} and on the desire function n_{ij} of vertex j , which can be included in the formed maximal clique if the ant is at vertex i . The desire function n_{ij} is the preference of the ant for selecting vertex j when it is at vertex i . In this study, the roulette principle was used for selection. Vertex j can be selected from the set $(V - V_{visited} - K_{1,...,k-1})_A$, where $V_{visited}$ represents vertices from parts of the graph visited during the creation of clique K_k , $K_{1,...,k-1}$ represents vertices

from cliques already created by the ant, and the A represents vertices available for selection. $G_{d,m}(E,V)$ is a d-partite graph with m vertices in each of its parts.

Algorithm 1. Ant_algorithm_1

```

General_Ant_Algorithm_1 ( $G_{d,m}(E,V)$ )
for each  $e_{ij} \in E$  do  $\tau_{ij} = \tau_{\max}$ ;
for cycle = 1 to lc (maximum number of cycles) do begin
     $i = 0$ ; ( $i$ —this is the vertex number in the first part of the graph)
    for nant = 1 to lm (lm—maximum number of ants) do begin
         $i = i + 1$ ;  $k = 0$ ;
        while  $k < m$  ( $m$ —number of vertices in graph part) do begin
             $k = k + 1$ ; (the number of constructed clique by ant, at the end
                        it should be equal to number of vertices  $m$  in graph parts)
             $K_k = \emptyset$ ; ( $K_k$ —the maximum clique that is under construction, now it is empty)
             $p = 0$ ;
            while  $p < d$  ( $d$ —number of parts) do begin
                 $p = p + 1$ ; ( $p$ —is used to count visited by an ant parts of the graph)
                select a next vertex  $v_j \in (V - V_{\text{visited}} - K_{1\dots k-1})_A$  with  $p_{ij} = \frac{\tau_{ij}^3 * \eta_{ij}}{\sum_j (\tau_{ij}^3 * \eta_{ij})}$ ;
                 $K_k = K_k \cup v_j$ ; (add selected vertex  $v_j$  to clique  $K_k$ )
                 $i = j$ ; (an ant goes to the vertex  $j$ )
            end (while  $p > d$ );
        end (while  $k > m$ );
        store the set_of_max_cliques with the smallest total weight  $W_K$  in a cycle;
    end (while nant > lm);
    calculate the  $d\tau = \frac{1}{\left(1 - \frac{(W_{Kg} - W_K)}{W_{Kg}}\right)}$ ;
    for all vertices  $\tau_{ij} = \rho * \tau_{ij}$ ;
    for the best set_of_max_cliques with the smallest total weight  $W_K$ , for each edge  $\tau_{ij} = \tau_{ij} + d\tau$ ;
    store the set_of_max_cliques with the smallest total weight under the variable  $W_{Kg}$  in cycles;
end (cycle > lc);
  
```

Ant_Algorithm_2 differs from Ant_algorithm_1 in that they are used to achieve different optimization goals. The goal of Ant_Algorithm_1 is to minimize W_{Kg} , and the goal of the Ant_Algorithm_2 is to maximize $(nK * W_K)_g$. W_{Kg} is the smallest sum of the weights of all maximum cliques from all cycles, while $(nK * W_K)_g$ is the greatest product of the number of maximum cliques and the sum of the weights of all maximum cliques from all cycles. Additionally, they differ in terms of their desire functions and the available vertices in next selection step when creating the maximum clique. In Ant_Algorithm_1, all vertices from the unvisited part of the graph are available, while in Ant_Algorithm_2, only these that are connected by edges to all of the vertices of the clique being built $(V - V_{\text{visited}})_A$, where A represents available vertices, are considered. Ant_Algorithm_2 is used to find the maximum number of maximum cliques in an incomplete, unweighted graph, and for this purpose, the parameter W_K is set to 1. Ant-Algorithm_2 is also used to find the maximum number of maximum cliques with the maximum sum of their weights in a noncomplete, weighted graph, where the parameter W_K is used to store the maximum sum of the cliques' weights. The parameter nK is used to store the maximum number of cliques.

Algorithm 2. Ant_algorithm_2

```

General_Ant_Algorithm_2 ( $G_{d,m} (E,V)$ )
for each  $e_{ij} \in E$  do  $\tau_{ij} = \tau_{\max}$ ;
for cycle = 1 to lc (lc—maximum number of cycles) do begin
     $i = 0$ ; ( $i$ —this is the vertex number in the first part of the graph)
    for nant = 1 to lm (lm—maximum number of ants) do begin
         $i = i + 1$ ;  $k = 0$ ;
        while  $k < m$  ( $m$ —number of vertices in graph part) do begin
             $k = k + 1$ ; (the number of constructed clique, at the end
                        it should be equal to number of vertices  $m$  in graph parts)
             $K_k = \emptyset$ ; ( $K_k$ —the maximum clique that is under construction, now it is empty)
             $p = 0$ ;
            while  $p < d$  (number of graph parts) do begin
                 $p = p + 1$ ; ( $p$ —is used to count visited by an ant parts of the graph)
                select a next vertex  $v_j \in (V - V_{visited} - K_{1\dots k-1})_A$  with  $p_{ij} = \frac{\tau_{ij}^3 * \eta_{ij}}{\sum_j (\tau_{ij}^3 * \eta_{ij})}$ ;
                 $K_k = K_k \cup v_j$ ; (add selected vertex  $v_j$  to clique  $K_k$ )
                 $i = j$ ; (an ant goes to vertex  $j$ )
            end (while  $p > d$ );
        end (while  $k > m$ );
        store the set_of_max_cliques with the largest number ( $nK * W_K$ ) in a cycle;
    end (while nant > lm);
    calculate the  $d\tau = 1 - \frac{((nK * W_K)_g - nK * W_K)}{(nK * W_K)_g}$ ;
    for all vertices  $\tau_{ij} = \rho * \tau_{ij}$ ;
    for the set_of_max_cliques with the largest number ( $nK * W_K$ ) in a cycle  $\tau_{ij} = \tau_{ij} + d\tau$ ;
    store the set_of_max_cliques with the largest number under  $(nK * W_K)_g$  in cycles;
end (cycle > lc);

```

5. Desire Functions

This section presents different desire functions for solving the problems considered.

5.1. Determining All Maximum Cliques with Minimum Sum of Their Weights in Complete Graph

The mechanism of the selection of the next vertex by ants during maximum clique creation is shown in Figure 2.

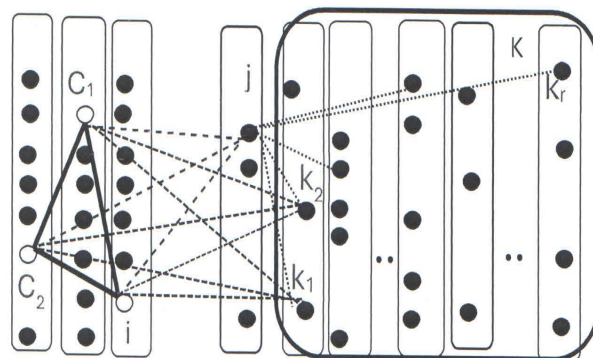


Figure 2. Looking for the next vertex in a weighted graph.

The ant is at vertex i . So far, let us assume that the ant has formed a clique consisting of vertices $\{c_1, c_2, i\}$ from three parts of the graph. Vertex j can be any vertex from the other unvisited parts of the graph, so the set of j vertices includes a subset V of k vertices, which are the vertices considered in the next step, which involves the selection of the next vertex to form a clique after selecting vertex j . Vertex j forms a clique with the already determined clique consisting of vertices $\{c_1, c_2, i\}$, so the cost of

including this vertex j in the formed clique K is computed as the sum of the edge weights through which this vertex j is connected to the already determined clique (see Equation (5)).

$$w_{Kj} = w_{ij} + w_{c1j} + w_{c2j} \quad (5)$$

The same applies to any vertex k , and the cost of its inclusion in the clique K is calculated using Equation (6).

$$w_{Kk} = w_{ik} + w_{c1k} + w_{c2k} \quad (6)$$

The weights of the edges between vertices i and j and between vertices j and k are denoted as w_{ij} and w_{jk} , respectively. In addition, the quantity $\min_{-}()$ represents the minimum values among the calculated values for all k vertices, and the quantity w_{Kk} is the weight of all edges connecting vertex k with the clique K being created. These quantities were assembled in various ways into functions, and on their basis, the following ant algorithms were established:

- ALG_1 is an algorithm with a desire function expressed as

$$\frac{1}{w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \min_{-}(w_{Kk})} \quad (7)$$

This is the best ant algorithm to date, and it is discussed and referred to as ALG_5 in [5]. In [2,3], ALG_1 outperforms all known ant algorithms, which is why it is used in this paper as a reference.

- ALG_2 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * w_{Kj}^2 * \min_{-}(w_{jk} * w_{Kk})} \quad (8)$$

- ALG_3 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * w_{Kj}^2 * \min_{-}(w_{jk} * (w_{Kk} + w_{jk}))} \quad (9)$$

- ALG_4 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \min_{-}(w_{jk} * w_{Kk})} \quad (10)$$

- ALG_5 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \min_{-}(w_{jk} * (w_{Kk} + w_{jk}))} \quad (11)$$

- ALG_6 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \min_{-}(w_{Kj} * (w_{Kk} * w_{jk}))} \quad (12)$$

- ALG_7 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * w_{Kj}^2 * \min_{-}(w_{Kj} * (w_{Kk} * w_{jk}))} \quad (13)$$

- ALG_8 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \min_{-}(w_{Kj} * w_{ij} * w_{Kk} * w_{jk})} \quad (14)$$

- ALG_9 is an algorithm with a desire function expressed as follows (new proposal):

$$\frac{1}{w_{ij}^2 * w_{Kj}^2 * \min_-(w_{Kj} * w_{ij} * w_{Kk} * w_{jk})} \quad (15)$$

5.2. Determining Maximum Number of Maximum Cliques in Unweighted Noncomplete Graph

The mechanism of the selection of the next vertex by ants during maximum clique creation is shown in Figure 3.

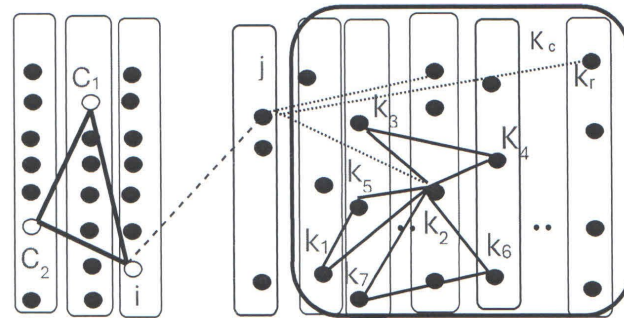


Figure 3. Looking for the next vertex in an unweighted graph.

The ant is at a vertex i and now has to choose from a set of j vertices (the set of j vertices contains a subset of k vertices), which are connected to all vertices of the clique formed so far. Let us assume it consists of vertices $\{c_1, c_2, i\}$. The k vertices that can be chosen after the selection of vertex j , and that therefore influence the selection of vertex j , are also taken into account. The selection of vertex j determines the choice of subsequent vertices, because after selecting vertex j , the set of available vertices consists of those connected to vertex j and the vertices from the formed clique. Therefore, choosing the next vertex k reduces the number of available vertices and hinders the achievement of a maximum clique of size d . It is possible that such a clique cannot be formed.

Each vertex j can create a different number of three-vertex cliques with vertices from other parts of the graphs, which will be denoted as $lk3wj$. Among the j vertices, we can choose the one that creates the largest number of three-vertex clicks with vertices from other parts of the graph, and this number will be denoted as max_lk3wj . Similarly, for k vertices, which are connected to j vertices, we can determine the maximum number of three-vertex cliques that each of them creates with vertices from other parts of the graph and this number will be denoted as max_lk3wk .

For vertex j , we can also sum up all of the three-vertex cliques that form k vertices neighboring vertex j , and the number of these three-vertex cliques is represented by $r, (\sum_{k=1}^r lk3wk)_j$. Of course, one of these sums will be the largest $max_(\sum_{k=1}^r lk3wk)$. All of these quantities influence the selection of vertex j and are combined in different ways; based on them, the following ant algorithms with different desire functions have been defined:

- ALGM_1 is an algorithm with a pheromone, but without a desire function.
- ALGM_2 is an algorithm with a desire function expressed by

$$\frac{1 + lk3wj}{1 + max_lk3wj} \quad (16)$$

The use of this ant algorithm to solve this problem is investigated for the first time in [4].

- ALGM_3 is an algorithm with a desire function expressed by

$$\frac{1 + lk3wj}{1 + max_lk3wj} * \frac{1 + lk3wk}{1 + max_lk3wk} \quad (17)$$

This is ALG_N4 [5], the best ant algorithm to date for solving this problem and was thus used as a reference in this study.

- ALGM_4 is an algorithm with a desire function expressed as follows (new proposal):

$$\left(\frac{1 + lk3w_j}{1 + \max_lk3w_j} * \frac{1 + (\sum_{k=1}^r lk3w_k)_j}{1 + \max_(\sum_{k=1}^r lk3w_k)} \right)^3 \quad (18)$$

- ALGM_5 is an algorithm with a desire function expressed as follows (new proposal):

$$\left(\frac{1 + lk3w_j}{1 + \max_lk3w_j} * \frac{1 + (\sum_{k=1}^r lk3w_k)_j}{1 + \max_(\sum_{k=1}^r lk3w_k)} * \max_lk3w_k \right)^3 \quad (19)$$

5.3. Determining the Maximum Number of Maximum Cliques with the Maximum Sum of Their Weights

To address the problem of determining the maximum number of cliques with the maximum sum of their weights in incomplete weighted graphs, ant algorithms are presented for the first time in this article. No ant algorithms for solving this problem have been published to date. The algorithms are as follows:

- ALGR_1 is the algorithm ALGM_5, which serves as a reference for this problem.
- ALGR_2 (new proposal) is as follows:

$$w_{ij}^2 * (w_{Kj} - w_{ij})^2 * \max_ (w_{Kj} * (w_{Kk} * w_{jk})) * \left(\frac{1 + lk3w_j}{1 + \max_lk3w_j} * \frac{1 + (\sum_{k=1}^r lk3w_k)_j}{1 + \max_(\sum_{k=1}^r lk3w_k)} \right)^3 \quad (20)$$

- ALGR_3 (new proposal) is as follows:

$$w_{ij}^2 * w_{Kj}^2 * \max_ (w_{Kj} * (w_{Kk} * w_{jk})) * \left(\frac{1 + lk3w_j}{1 + \max_lk3w_j} * \frac{1 + (\sum_{k=1}^r lk3w_k)_j}{1 + \max_(\sum_{k=1}^r lk3w_k)} \right)^3 \quad (21)$$

6. Experiments

Various algorithms were tested for the three optimization problems considered.

6.1. All Maximum Cliques with Minimum Sum of Their Weights in Complete Weighted Graph

ALG_1 to ALG_9 were compared with each other, and the results of these comparisons are included in Tables 1 and 2. These tables contain the averages of 10 measurements. The obtained results are the averages of the smallest sum of the weights of all determined maximal cliques, and these averages are given in thousands. The weight of a single edge is generated as a number between 1 and 100. The parameters that changed during the tests were as follows: m —the number of vertices in any part of the graph; d —the number of graph parts; lm —the number of ants; lc —the number of cycles; ρ —the pheromone evaporation coefficient. The use of different desire functions in the ant algorithms allowed us to obtain better results than those published to date due to the minimum total weight of all maximum cliques. The best algorithms were found to be ALG_6 and ALG_7.

Table 1. Sumed weights of maximum cliques: $d = 25$, $lm = 10$, $lc = 30$, $\rho = 0.995$.

m:	20	25	30	35	40
ALG_1	269.1	336.0	402.4	469.0	534.4
ALG_2	267.9	332.4	397.8	463.5	528.7
ALG_3	267.3	332.4	398.9	463.8	528.7
ALG_4	266.9	331.5	397.9	463.3	529.1
ALG_5	266.8	332.8	397.7	462.8	528.9
ALG_6	264.1	328.8	393.1	457.1	520.1
ALG_7	264.1	328.3	392.9	456.7	521.6
ALG_8	265.8	331.2	396.6	460.9	525.4
ALG_9	266.6	331.4	396.4	460.7	525.4

Table 2. Sumed weights of maximum cliques: $m = 20$, $lm = 10$, $lc = 30$, $\rho = 0.995$.

d:	20	25	30	35	40
ALG_1	206.6	336.0	496.4	689.0	912.6
ALG_2	205.0	332.4	492.3	684.7	907.2
ALG_3	204.4	332.5	492.8	684.5	907.1
ALG_4	204.6	331.5	492.6	683.5	906.6
ALG_5	204.2	332.8	492.9	683.4	906.9
ALG_6	201.7	328.8	487.1	677.2	897.8
ALG_7	200.9	328.3	487.3	677.4	899.2
ALG_8	203.1	331.2	490.0	681.6	902.1
ALG_9	203.3	331.4	490.6	681.0	902.7

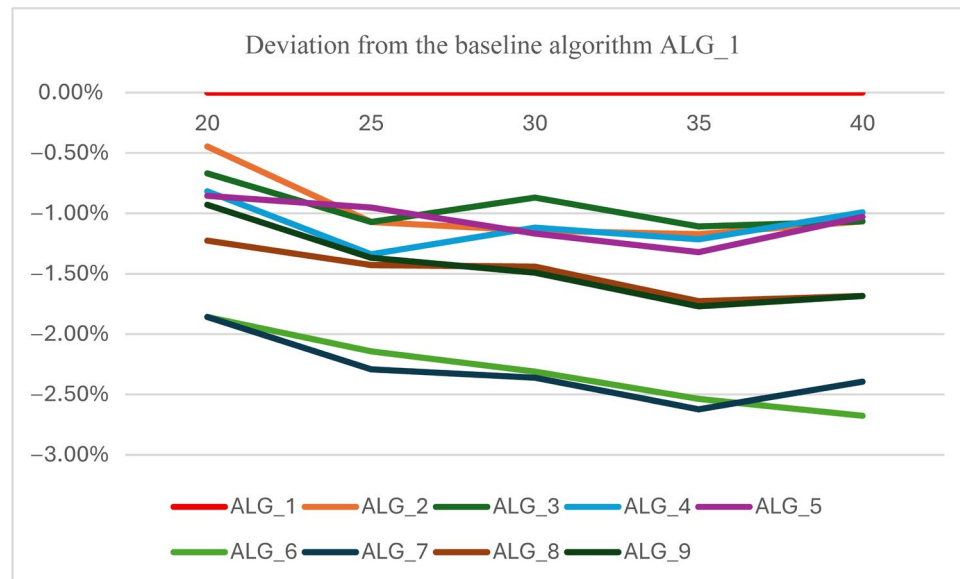
Table 1 shows the results obtained when the number of vertices in each part of the graph m changes while the number of graph parts d , the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. It can be seen that ALG_6 and ALG_7 show increasingly better results compared to the other algorithms as the number of vertices in the graph section increase.

Figure 4 presents the deviation in the results obtained by the other proposed algorithms in relation to the baseline algorithm ALG_1 [5]. The deviation is calculated according to the formula

$$\frac{W_{ALG_1} - W_{ALG_x}}{W_{ALG_1}} \quad (22)$$

where

W_{ALG_1} represents the minimum sum of the weights of all maximum clicks obtained from ALG_1; W_{ALG_x} represents the minimum sum of the weights of all maximum clicks obtained from the remaining algorithms (ALG).

**Figure 4.** Sumed weights of maximum cliques: m changes, $d = 25$, $lm = 10$, $lc = 30$, $\rho = 0.995$.

It is clear that ALG_6 and ALG_7 achieve increasingly better results compared to the original algorithm ALG_1 from the literature [5] as the number of vertices in each part of the graph increases. Therefore, when the search space becomes larger, ALG_6 and ALG_7 perform better than the other algorithms.

Table 2 shows the results obtained when the number of vertices in each part of the graph m is constant; the number of graph parts d changes; and the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. It can be seen that ALG_6 and ALG_7 show increasingly better results compared to the other algorithms as the number of graph sections increases.

Figure 5 also shows how the deviation in the results obtained by the other proposed algorithms changes in relation to the base algorithm ALG_1 [5] as the number of graph parts increases. Here it can be seen that the increase in the dimensions of the maximum cliques determined reduces the benefits gained by the proposed ant colony algorithms compared to those gained by the previously published algorithm [5].

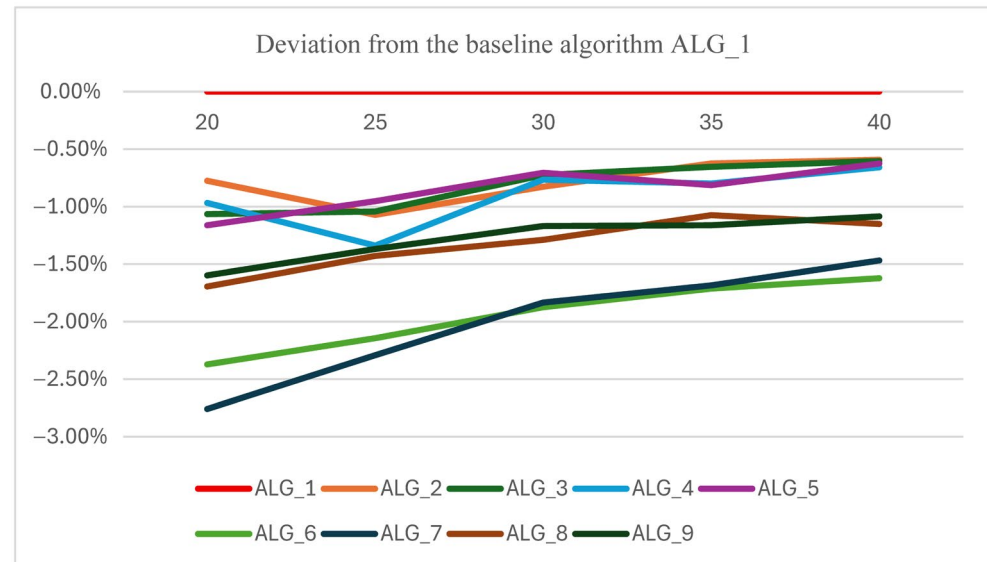


Figure 5. Sumed weights of maximum cliques: d changes, $m = 20$, $lm = 10$, $lc = 30$, $\rho = 0.995$.

6.2. Maximum Number of Maximum Cliques in Noncomplete Unweighted Graph

ALGM_1 to ALGM_5 were compared with each other, and the results of these comparisons are shown in in Tables 3–5. These tables contain the averages of 10 measurements. The obtained results are the averages of the largest number of maximum cliques obtained. The parameters that changed during the tests were as follows: q —the density of the graph; m —the number of vertices in one part of the graph; d —the number of graph parts; lm —the number of ants; lc —the number of cycles; ρ —the pheromone evaporation coefficient. The use of different desire functions in the ant algorithms allowed us to obtain better results than those published to date, as it resulted in the largest number of maximum cliques obtained. ALGM_4 was found to be the best algorithm.

Table 3. Number of maximum cliques received: $d = 11$, $m = 60$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

q :	0.6	0.65	0.7	0.75	0.8
ALGM_1	3.7	4.5	24.6	39.8	49.7
ALGM_2	4.7	12.2	27.3	41.1	51.3
ALGM_3	4.0	13.1	28.7	43.3	51.9
ALGM_4	6.6	16.1	31.3	44.3	53.0
ALGM_5	6.1	15.5	30.5	43.8	52.0

Table 4. Number of maximum cliques: $m = 50$, $q = 0.75$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

d :	7	9	11	13	15
ALGM_1	46.5	40.6	30.3	17.1	7.6
ALGM_2	46.7	41.4	32.0	19.4	8.6
ALGM_3	47.3	42.5	33.5	20.6	9.1
ALGM_4	47.6	43.0	35.5	22.9	11.1
ALGM_5	47.6	42.4	33.8	21.3	10.1

Table 5. Number of maximum cliques: $d = 11$, $lc = 30$, $lm = 10$, $\rho = 0.995$, $q = 0.75$.

m:	20	30	40	50	60
ALGM_1	6.4	12.8	21.5	30.3	39.8
ALGM_2	7.0	14.5	22.6	32.0	41.1
ALGM_3	7.3	15.0	23.8	33.5	43.3
ALGM_4	8.6	16.4	25.4	35.5	44.3
ALGM_5	7.7	15.3	24.7	33.8	43.8

Table 3 shows the results obtained when the graph density q changes while the number of vertices in each part of the graph m , the number of graph parts d , the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. ALGM_4 resulted in the largest number of maximum cliques compared with the others algorithms.

Figure 6 shows the results of the algorithms with varying density levels of the incomplete graph. ALGM-1 is the ant algorithm without a desirability function, ALGM_2 is the algorithm proposed in [4], and ALGM_3 is the algorithm proposed in [5]. ALGM_4 and ALGM_5 are the new algorithms proposed in this paper and they perform better than those previously published.

Table 4 shows the results obtained when the graph density q and the number of vertices in each part of the graph m are constant; the number of graph parts d changes; and the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. The ALGM_4 resulted in the largest number of maximum cliques compared with the other algorithms.

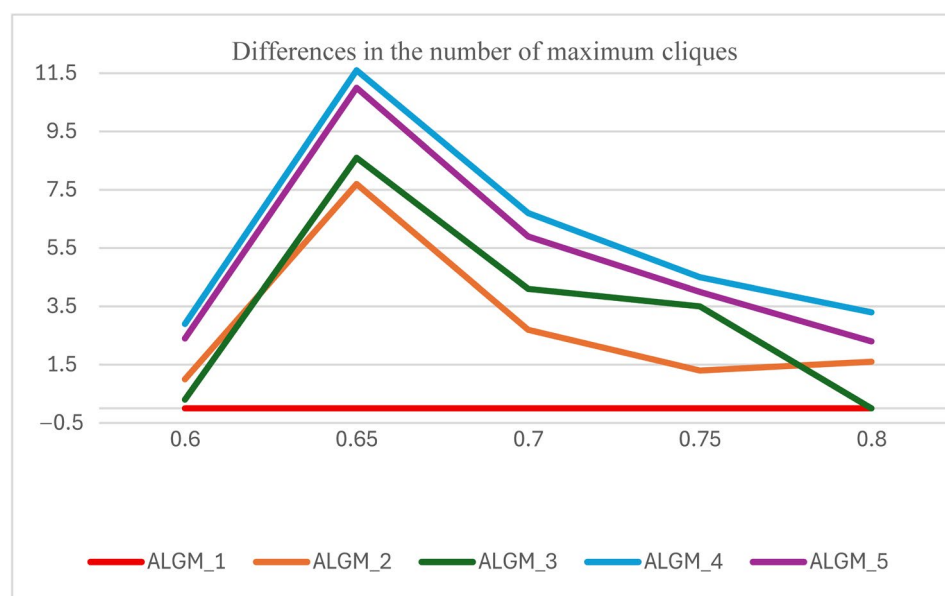


Figure 6. Number of maximum cliques: q changes, $d = 11$, $m = 60$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

Figure 7 shows the results obtained when the number of vertices in each part of the graph ($m = 50$) and the graph density ($q = 0.75$) are constant while the number of graph parts changes. When the graph is not complete and the number of graph parts increases, the number of maximum cliques determined decreases. ALGM_3 is the best ant colony algorithm published to date, but it is clear that the proposed ALGM_4 and ALGM-5 allow for the determination of a greater number of maximum cliques.

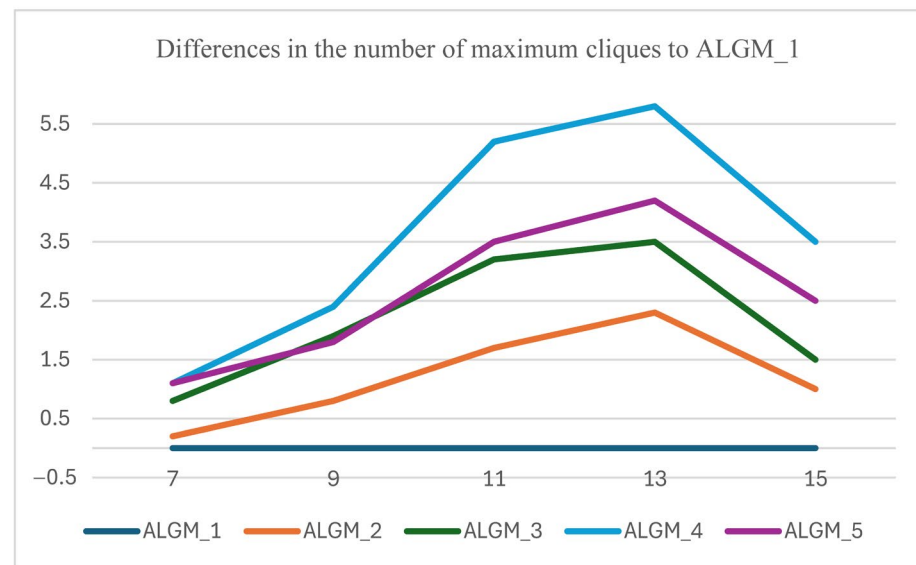


Figure 7. Number of maximum cliques: d changes, $m = 50$, $q = 0.75$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

Table 5 shows the results obtained when the graph density q is constant; the number of vertices in each part of the graph m changes; and the number of graph parts d , the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. ALGM_4 results in the largest number of maximum cliques compared with the other algorithms.

In Figure 8, with a graph density of $q = 0.75$ and a constant number of graph parts ($d = 11$), it can be seen that an increase in the number of vertices in each part of the graph contributes to an increase in the number of maximum cliques determined and that this is a constant upward trend.

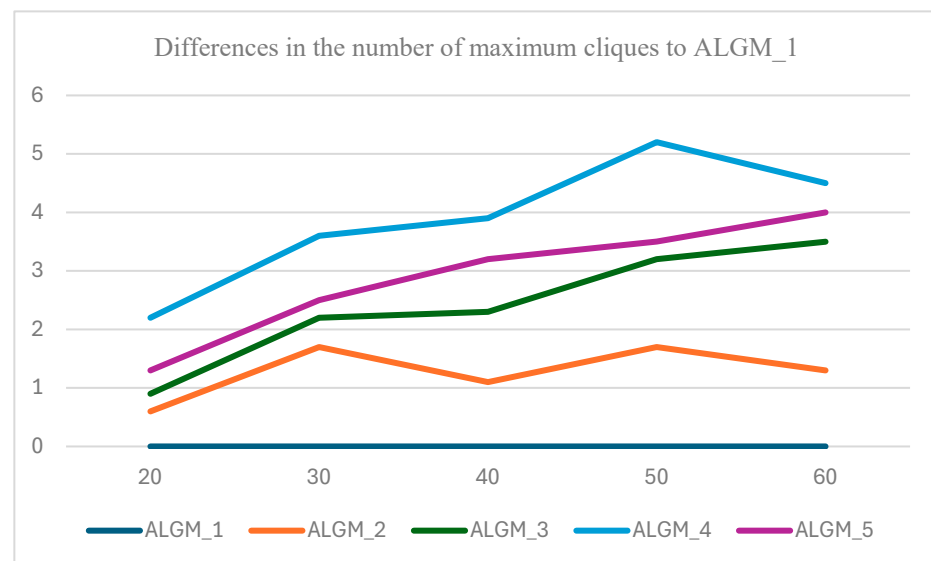


Figure 8. Number of maximum cliques: m changes, $d = 11$, $lc = 30$, $lm = 10$, $\rho = 0.995$, $q = 0.75$.

6.3. Maximum Number of Maximum Cliques in Noncomplete Weighted Graph

ALGR_1 to ALGR_3 were compared with each other, and the results of these comparisons are shown in in Tables 6–8. These tables contain the averages of 10 measurements. The obtained results are the averages of the largest number of maximum cliques obtained nK and of the maximum sum of the maximum clique weights W . The parameters that changed during the tests were as follows: q —the density of the graph; m —the number of vertices in one part of the graph; d —the number of graph parts; lm —the number of ants; lc —the number of cycles; ρ —the pheromone evaporation coefficient.

Table 6. Weights and number of maximum cliques: $d = 10, m = 40, lc = 30, lm = 10, \rho = 0.995$.

q :		0.7	0.75	0.8	0.85	0.9
ALGR_1	W	49,897	67,663	80,004	85,353	89,199
	nK	22.1	29.6	35.1	37.9	39.1
ALGR_2	W	86,595	100,749	111,913	118,064	123,668
	nK	16.2	26.0	33.0	36.8	38.9
ALGR_3	W	87,425	99,223	110,504	117,409	123,102
	nK	16.5	26.4	33.0	37.0	39.0

Table 7. Weights and number of maximum cliques: d changes, $m = 40, q = 0.8, lc = 30, lm = 10, \rho = 0.995$.

d :		6	8	10	12	14
ALGR_1	W	29,930	53,423	80,004	99,542	102,656
	nK	39.6	37.7	35.1	30.3	22.4
ALGR_2	W	45,035	77,218	111,913	151,287	179,214
	nK	39.2	36.4	33.0	25.9	18.3
ALGR_3	W	45,550	76,077	110,504	145,867	178,998
	nK	39.1	37.0	33.0	26.1	18.5

Table 8. Weights and number of maximum cliques: $d = 10, lc = 30, lm = 10, \rho = 0.995, q = 0.8$.

m :		10	20	40	50	60
ALGR_1	W	13,070	34,569	85,353	102,172	124,885
	nK	5.7	15.3	37.9	45.0	54.7
ALGR_2	W	21,276	49,633	118,064	141,106	172,215
	nK	4.6	13.3	36.8	43.6	53.0
ALGR_3	W	18,417	49,945	117,409	142,883	172,201
	nK	5.4	13.7	37.0	43.5	52.9

Table 6 shows the results obtained when the graph density q changes while the number of vertices in each part of the graph m , the number of graph parts d , the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. ALGR_2 is slightly better than ALGR_3 in terms of the maximum sum of clique weights and slightly worse in terms of the maximum number of cliques.

Figure 9 shows the deviation in the sum of the maximum cliques and in the number of maximum cliques determined with varying graph density, fixed numbers of vertices in each part of the graph, and a constant number of graph parts. The deviation in the number of maximum cliques determined is defined as follows:

$$\frac{nK_{ALGR_1} - nK_{ALGR_x}}{W_{ALGR_1}} \quad (23)$$

where

nK_{ALGR_1} represents the maximum number of all maximum clicks obtained by ALGR_1;
 nK_{ALGR_x} represents the maximum number of all maximum clicks obtained by the remaining ALGR algorithms.

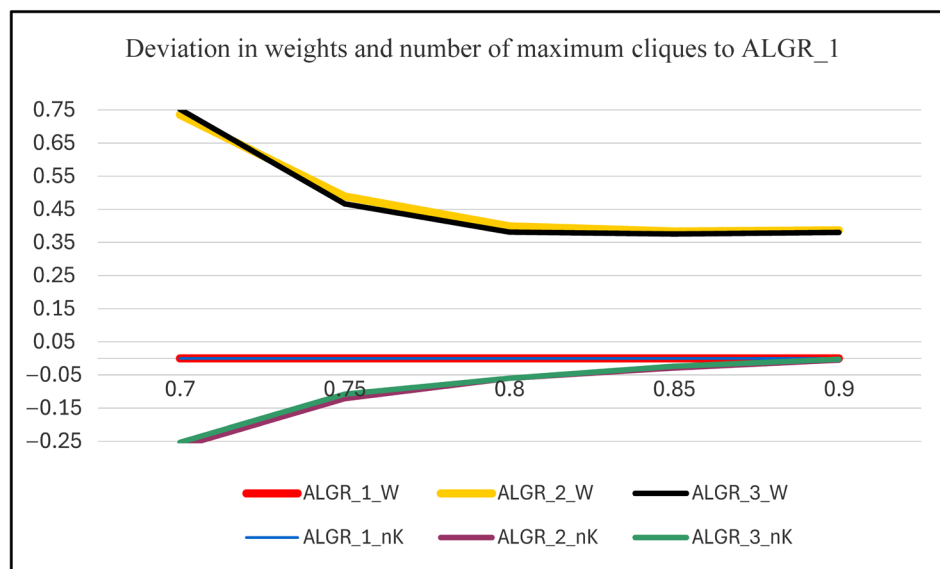


Figure 9. Weights and number of maximum cliques: q changes, $d = 10$, $m = 40$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

Figure 9 shows that ALGR_2 and ALGR_3 operate similarly, and with an increase in graph density to $q = 0.8$, they determine an increasingly larger number of maximum cliques, while the sum of the weights of these cliques decreases. However, above a density of $q = 0.8$, these two algorithms demonstrate a greater sum of the weights of maximum cliques compared to ALGR_1, while the number of cliques determined converges to the number of maximum cliques determined by ALGR_1.

Table 7 shows the results obtained when the graph density q and the number of vertices in each part of the graph m are constant; the number of graph parts d changes; and the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. ALGR_2 is slightly better than ALGR_3 in terms of the maximum sum of clique weights and slightly worse in terms of the maximum number of cliques.

Figure 10 shows that the change in the number of graph parts affects the behavior of the algorithms. ALGR_2 and ALGR_3 operate similarly above $d = 10$, and as the number of graph parts increases, the number of maximum cliques determined by them decreases, while the sum of the weights of the maximum cliques determined increases.

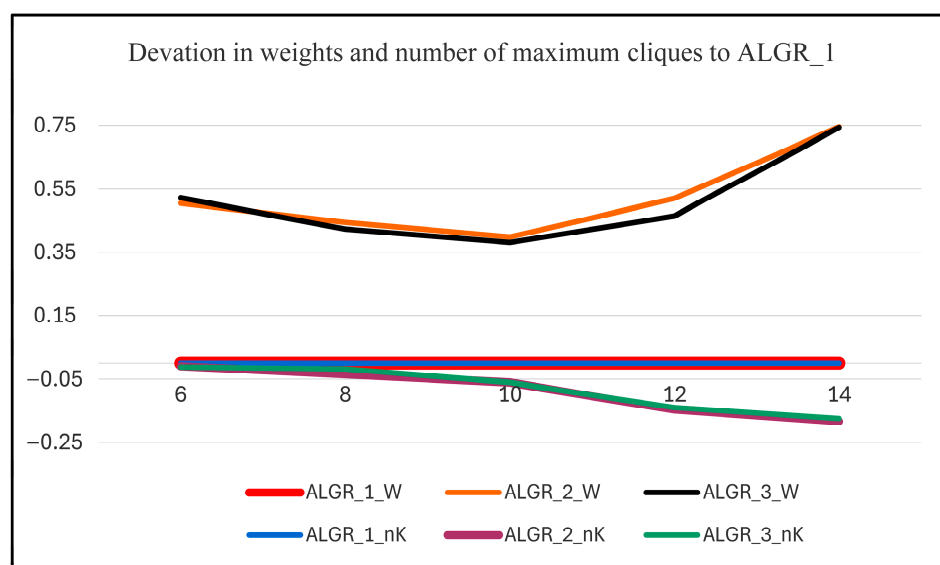


Figure 10. Weights and number of maximum cliques: d changes, $m = 40$, $q = 0.8$, $lc = 30$, $lm = 10$, $\rho = 0.995$.

Table 8 shows the results obtained when the graph density q is constant; the number of vertices in each part of the graph m changes; and the number of graph parts d , the number of ants lm , the number of cycles lc , and the evaporation rate ρ are constant. ALGR2 is slightly better than ALGR3 in terms of the maximum sum of clique weights and slightly worse in terms of the maximum number of cliques.

Figure 11 shows that an increase in the number of vertices in each part of the graph above $m = 40$ does not impact the differentiation in the performance of the algorithms. ALGR_2 and ALGR_3 perform similarly to ALGR_1 when determining the maximum number of maximum cliques, while in terms of the sum of the weights of the maximum cliques, ALGR_2 and ALGR_3 allow for larger sums to be obtained compared to ALGR_1. When the number of vertices in each part of the graph is below $m = 40$, it is evident that with an increase in the number of maximum cliques determined, the relative differentiation among the algorithms decreases when determining the sum of the weights of the maximum cliques.

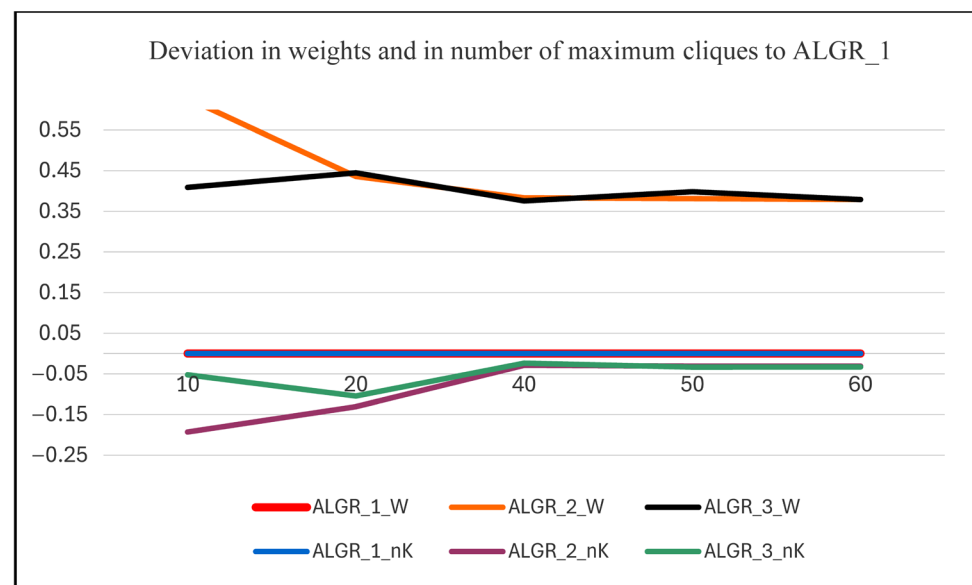


Figure 11. Weights and number of maximum cliques: m changes, $d = 10$, $lc = 30$, $lm = 10$, $\rho = 0.995$, $q = 0.8$.

7. Conclusions

In this study, many ant algorithms were examined in complete and incomplete d-partite graphs. All of the newly proposed ant algorithms for weighted complete d-partite graphs are better than those reported in the literature to date, and amongst them, ALG_6 and ALG_7 are the best in terms of determining all maximum cliques with the minimum sum of weights. Regarding the problem of determining the maximum number of maximum cliques in incomplete d-partite graphs, both of the ant algorithms proposed in this article are better than previously published ant algorithms, and our tests indicated that ALGM_4 are the best. Regarding the problem of determining all maximum cliques in incomplete d-partite graphs, all of the presented ant algorithms are proposed for the first time. The best amongst them are ALGR2 and ALGR_3. All of these new algorithms can also be applied to larger problems. Future research will focus on dealing with missing vertices in d parts of the graph. The developed algorithms can be applied to determine maximum cliques, for example, taking into account different time points or positions in space represented by parts of a graph, both with complete and incomplete information about the objects. In visual systems, these objects occur in film frames at different moments in time, and the edges of a graph or the absence of edges can represent similarity between objects or a complete lack of information about similarity. In the case of positions in space, the absence of edges may indicate a lack of connection or a prohibited direct connection, for example, between elements in electronic systems, or can indicate problems when optimizing paths for thread spools in the textile industry.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the author.

Conflicts of Interest: The author declares no conflict of interest.

References

1. Bozdogan, A.O.; Egemen, A.E.; Efe, M. Performance analysis of swarm optimization approaches for the generalized assignment problem in multi-target tracking applications. *Turk. J. Electr. Eng. Comput. Sci.* **2010**, *18*, 1059–1078. [\[CrossRef\]](#)
2. Bozdogan, A.O.; Efe, M. Ant Colony Optimization Heuristic for the multidimensional assignment problem in target tracking. In Proceedings of the IEEE National Radar Conference 2008, Rome, Italy, 26–30 May 2008; pp. 1–6.
3. Bozdogan, A.O.; Efe, M. Improved assignment with ant colony optimization for multi-target tracking. *Expert Syst. Appl.* **2011**, *38*, 9172–9178. [\[CrossRef\]](#)
4. Schiff, K. Ant Colony Optimization Algorithm for Object Identification in Multi-cameras Video Tracking Systems. In *New Advances in Dependability of Networks and Systems*; Zamojski, W., Mazurkiewicz, J., Sugier, J., Walkowiak, T., Kacprzyk, J., Eds.; DepCoS-RELCOMEX 2022, Lecture Notes in Networks and Systems; Springer: Cham, Switzerland, 2022; Volume 484.
5. Schiff, K. Ant algorithms for finding weighted and unweighted maximum cliques in d-division graphs. *Int. J. Electron. Telecommun.* **2024**, *70*, 381–387. [\[CrossRef\]](#)
6. Emami, P.; Pardalos, P.M.; Eleftheriadou, L.; Ranka, S. Machine Learning Methods for Data Association in Multi-Object Tracking. *ACM Comput. Surv. CSUR* **2020**, *53*, 69. [\[CrossRef\]](#)
7. Dehghan, A.; Assari, S.M.; Shah, M. GMMCP tracker: Globally optimal Generalized Maximum Multi Clique problem for multiple object tracking. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, 7–12 June 2015; pp. 4091–4099.
8. Kumar, R.; Charpiat, G.; Thonnat, M. Multiple Object Tracking by Efficient Graph Partitioning. In *Computer Vision—ACCV 2014*; Cremers, D., Reid, I., Saito, H., Yang, M.H., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2014; Volume 9006.
9. Wei, L.; Xingwei, L. Multiple object tracking based on modified algorithm of GMMCP tracker. In Proceedings of the 2016 IEEE International Conference on Signal and Image Processing (ICSIP), Beijing, China, 13–15 August 2016; pp. 11–15.
10. Jiang, K.; Ma, T.; Hu, Y. GMMCP Based Multi Target Tracking Algorithm with Missing Measurements. In Proceedings of the 9th International Conference on Mechanical and Electronics Engineering (ICMEE), Xi'an, China, 17–19 November 2023; pp. 307–310.
11. Grunert, T.; Irnich, S.; Zimmermann, H.J.; Schneider, M.; Wulforst, B. Finding all k-cliques in k-partite graph, an application in textile engineering. *Comput. Oper. Res.* **2002**, *29*, 13–31. [\[CrossRef\]](#)
12. Mirghorbani, M.; Krokhmal, P. On finding k-cliques in k-partite graphs. *Optim. Lett.* **2013**, *7*, 1155–1165. [\[CrossRef\]](#)
13. Wen, L.; Lei, Z.; Lyu, S.; Li, S.Z.; Yang, M. Exploiting hierarchical dense structures on hyper-graphs for multi-object tracking. *IEEE Trans. Pattern Anal. Mach. Intell.* **2016**, *38*, 1983–1996. [\[CrossRef\]](#) [\[PubMed\]](#)
14. Wang, H.; Kirubarajan, T.; Bar-Shalom, Y. Precision large scale air traffic surveillance using IMM/assignment estimators. *Trans. Aerosp. Electron. Syst.* **1999**, *35*, 255–266. [\[CrossRef\]](#)
15. Gabrovsek, B.; Novak, T.; Povh, J.; Rupnik Poklukar, D.; Zerovnik, J. Multiple Hungarian method for k-assignment problem. *Mathematics* **2020**, *8*, 2050. [\[CrossRef\]](#)
16. Deb, S.; Yeddapanudi, M.; PattiPati, K.; Bar-Shalom, T. A generalized S-D assignment algorithm for multi-sensor-multitarget state estimation. *IEEE Trans. Aerosp. Electron. Syst.* **1999**, *33*, 523–538.
17. Feremans, C.; Labbe, M.; Laportee, G. Generalized network design problem. *Eur. J. Oper. Res.* **2003**, *148*, 1–13. [\[CrossRef\]](#)
18. Koster, A.M.C.A.; Hoesel, S.P.M.; Kolen, A.W.J. The partial constraint satisfaction problem: Facets and lifting theorems. *Oper. Res. Lett.* **1998**, *23*, 89–97. [\[CrossRef\]](#)
19. Zamir, A.R.; Dehghan, A.; Shah, M. GMPC—Tracker: Global Multi-object Tracking Using Generalized Minimum Clique Problem. In Proceedings of the Computer Vision—European Conference on Computer Vision 2012, Florence, Italy, 7–13 October 2012; pp. 343–356.
20. Andriyenko, A.; Schindler, K. Multi-target Tracking by Continuous Energy Minimization. In Proceedings of the Conference on Computer Vision and Pattern Recognition 2011, Colorado Springs, CO, USA, 20–25 June 2011; pp. 1265–1272.
21. Andriyenko, A.; Schindler, K.; Roth, S. Discrete-Continuous Optimization for Multi-Target Tracking. In Proceedings of the Conference on Computer Vision and Pattern Recognition 2012, Providence, RI, USA, 16–21 June 2012; pp. 1926–1933.

22. Liang, Y.; Lu, X.; He, Z.; Zheng, Y. Multiple object tracking by reliable tracklets. *Signal Image Video Process.* **2019**, *13*, 823–831. [[CrossRef](#)]
23. Berclaz, J.; Fleuret, F.; Turetken, E.; Fua, P. Multiple object tracking using k-shortest paths optimization. *IEEE Trans. Pattern Anal. Mach. Intell.* **2011**, *33*, 1806–1819. [[CrossRef](#)] [[PubMed](#)]
24. Zhang, L.; Li, Y.; Nevatia, R. Global data association for multi-object tracking using network flows. In Proceedings of the 2008 IEEE Conference on Computer Vision and Pattern Recognition, Anchorage, AK, USA, 23–28 June 2008; pp. 1–8.
25. Pirsiavash, H.; Ramanan, D.; Fowlkes, C.C. Globally-optimal greedy algorithms for tracking a variable number of objects. In Proceedings of the 2011 IEEE Conference on Computer Vision and Pattern Recognition, Colorado Springs, CO, USA, 20–25 June 2011; pp. 1201–1208.
26. Lenz, P.; Geiger, A.; Urtasun, R. Follow Me: Efficient online min-cost flow tracking with bounded memory and computation. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) 2015, Santiago, Chile, 7–13 December 2015.
27. Chari, V.; Lacoste-Julien, S.; Laptev, I.; Sivic, J. On pairwise costs for network flow multi-object tracking. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) 2015, Boston, MA, USA, 7–12 June 2015; pp. 5537–5545.
28. Ristani, E.; Tomasi, C. Tracking multiple people online and in real time. In *Asian Conference on Computer Vision*; Springer: Cham, Switzerland, 2014; pp. 444–459.
29. Tang, S.; Andres, B.; Andriluka, M.; Schiele, B. Subgraph Decomposition for Multi-Target Tracking. In Proceedings of the Conference on Computer Vision and Pattern Recognition 2015, Boston, MA, USA, 7–12 June 2015; pp. 5033–5041.
30. Morefield, C.L. Application of 0–1 integer programming to multi-target tracking problems. *IEEE Trans. Autom. Control* **1971**, *22*, 302–312. [[CrossRef](#)]
31. Yoon, K.; Kim, D.Y.; Yoon, Y.C.; Jeon, M. Data association for Multi-Object Tracking via Deep Neural Networks. *Sensors* **2019**, *19*, 559. [[CrossRef](#)] [[PubMed](#)]
32. Lee, B.; Erdenee, E.; Jin, S.; Rhee, P.K. Efficient object detection using convolutional neural network-based hierarchical feature modeling. *Image Video Proc.* **2016**, *10*, 1503–1510. [[CrossRef](#)]
33. Joelianito, E.; Wiranto, I. An application of Ant Colony Optimization, Kalman Filter and Artificial Neural Network for Multiple Target Tracking Problems. *Int. J. Artif. Intell.* **2011**, *7*, 384–400.
34. Chen, G.; Hong, L.A. genetic based multi-dimensional data association algorithm for multi sensor multi target tracking. *Math. Comput. Models* **1997**, *26*, 57–69. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.